typesense

Tips for Locale-Specific Search

Copy Markdown

In this article, we'll talk about how to handle locale-specific text in your Typesense collections. We'll cover:

- Basic Locale Configuration
- Default Behavior and English Text
- Language Code Support
- Commonly Used Languages
- Language-Specific Features
- Advanced Tokenization Options
- Best Practices

Basic Locale Configuration

To enable language-specific text handling, specify the locale when defining your collection's schema using the `locale` parameter in the field definition:

```json
{
  "name": "posts",
  "fields": [
    { "name": "title", "type": "string", "locale": "vi" },
    { "name": "description", "type": "string", "locale": "en" }
  ]
}
```

Each field can have its own locale setting, allowing you to handle multilingual content within the same document.

Default Behavior and English Text

When no locale is specified for a field, Typesense treats it as English (`en`) by default. This has important implications:

- For non-English locales, diacritics are preserved due to ICU (International Components for Unicode) tokenization

This affects search behavior in important ways:

- When searching fields with preserved diacritics, an exact match with accents will be prioritized
- Due to typo tolerance, if no exact accented match is found, non-accented versions will be matched
- For optimal matching, consider your users' likely search patterns when deciding whether to preserve diacritics

Language Code Support

Typesense supports any valid two-letter ISO 639-1 language code . This means you're not limited to just the commonly documented languages - you can use any standard language code like:

- `vi` for Vietnamese
- `id` for Indonesian
- `ms` for Malay
- `nl` for Dutch
- `pl` for Polish
- etc.

The locale parameter accepts any valid ISO 639-1 code and will use the appropriate ICU rules for that language.

How It Works

When you specify a locale:

1. Typesense uses ICU libraries for language processing
2. The two-letter code is passed directly to ICU's locale handler
3. Appropriate language rules are applied for tokenization and text processing

Setting the Locale

```
      { "name": "title", "type": "string", "locale": "ANY-VALID-ISO-CODE" }
    ]
}
```

Where `ANY-VALID-ISO-CODE` can be any standard two-letter language code.

Commonly Used Languages

While any ISO 639-1 code works, here are some commonly used languages with their codes:

Language	Code	Notes
Arabic	`ar`	Right-to-left script
Chinese	`zh`	Supports both simplified and traditional
Dutch	`nl`	
English	`en`	Default if no locale specified
French	`fr`	
German	`de`	
Hindi	`hi`	
Indonesian	`id`	
Italian	`it`	
Japanese	`ja`	
Korean	`ko`	
Malay	`ms`	
Polish	`pl`	
Portuguese	`pt`	
Russian	`ru`	Cyrillic script
Greek	`el`	Cyrillic script

Spanish	es	
Thai	th	
Turkish	tr	
Vietnamese	vi	

Language-Specific Features

Different language families receive specialized handling:

Script-Based Features

- **CJK Languages** (Chinese, Japanese, Korean)
  - Word segmentation without spaces
  - Character variant handling
- **Right-to-Left Scripts** (Arabic, Hebrew)
  - Proper text direction handling
  - Special character normalization
- **Languages with Special Characters**
  - Diacritic handling
  - Special character normalization
  - Proper collation

Word Segmentation

Some languages receive special word segmentation handling:

- **Thai**: No spaces between words, requires special breaking rules
- **Japanese**: Mixed kanji-kana text segmentation
- **Vietnamese**: Proper handling of tone marks and compounds
- **Chinese**: Character-based segmentation and variant handling

typesense

⌘ K

Advanced Tokenization Options

Custom Tokenization with `pre_segmented_query`

For languages with complex word boundaries or when you need more control over tokenization, Typesense offers the `pre_segmented_query` parameter. This feature allows you to:

1. Use your own tokenizer for both indexing and querying
2. Have Typesense simply split on spaces rather than applying its default tokenization rules

This is particularly useful for:

- Chinese and other CJK languages where ML-based word splitting might be more accurate
- Languages with complex compound words
- Special domain-specific tokenization needs

Example usage:

```json
{
  "q": "pre tokenized query",
  "pre_segmented_query": true
}
```

When using this feature, ensure that:

- Your indexing tokenization matches your query tokenization
- Tokens are space-separated in both indexed content and queries
- You maintain consistency in your tokenization approach

Best Practices

1. **Always Specify the Locale** if it's not an English field.

```json
{
  "fields": [
    { "name": "title_vi", "type": "string", "locale": "vi" },
    { "name": "title_th", "type": "string", "locale": "th" }
```

2. **Field Naming Convention** Include the language code in field names for multilingual content:

```json
{
  "title_en": "English Title",
  "title_vi": "Tiêu đề tiếng Việt",
  "title_th": "ชื่อเรื่องภาษาไทย"
}
```

3. **Consider Diacritic Handling**

   For European languages, consider whether your users are likely to search with or without diacritics.

4. **Custom Tokenization**

   Consider using `pre_segmented_query` for languages where default tokenization might not be optimal.

Edit page

Last Updated: 12/2/2024, 1:28:49 PM

← Tips for Searching Common Types of Data

Tips for Filtering Data in Typesense →